Génie Logiciel

Dr. Meryem Janati Idrissi

Master Sciences du Numérique et Intelligence Artficielle
Université Mohammed V de Rabat
Année Universitaire : 2024/2025

Chapitre III - Modélisation De Processus De Développement Logiciel

Modèles traditionnels

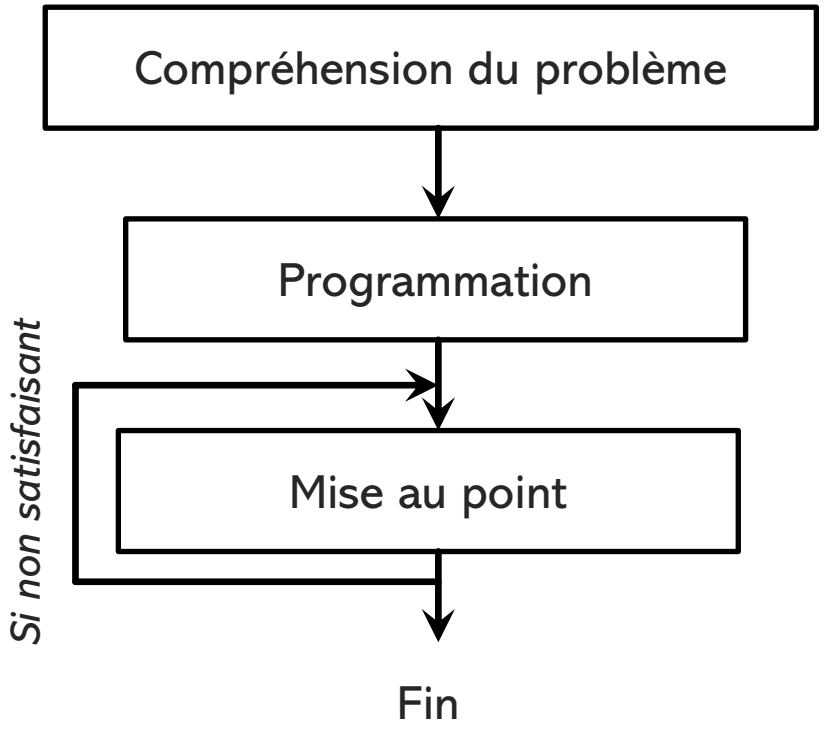
جامعة محمد الخامس بالرباط
Université Mohammed V de Rabat

Définition

- La modélisation de développement logiciel aide les développeurs à sélectionner la stratégie pour développer le produit logiciel. Ainsi, chaque modélisation de développement possède ses outils propres, ses méthodes et ses procédures qui permettent d'exprimer clairement et définissent le cycle de vie de développement du logiciel.
- Il s'avère de nos jours, que nous ne pouvons plus avoir une démarche unique dans le développement de projets informatiques, mais qu'il faut construire le découpage temporel en fonction des caractéristiques de l'entreprise et du projet. On s'appuie pour cela sur des découpages temporels génériques, appelés modèles de développement (process models) ou modèles de cycle de vie d'un projet informatique. Les principaux modèles sont :
    - le modèle du code-and-fix ;
    - le modèle de la transformation automatique ;
    - le modèle de la cascade ;
    - le modèle en V ;
    - e modèle par incrément;
    - le modèle par prototypage ;
    - le modèle de la spirale.

Le Modèle Du Code-and-fix

C’est un modèle qui repose sur la possibilité d’une détermination facile des besoins : une première phase de Compréhension du problème est suivie d’une phase de Programmation ; puis une phase de Mise au point, parfois en collaboration avec l'utilisateur du futur système, est répétée jusqu’à l’atteinte du résultat visé.



Le Modèle Du Code-and-fix

Le modèle Code-and-Fix consiste à :

- Commencer directement à coder, sans analyse détaillée des besoins ni conception.
- Tester après coup (souvent de manière informelle).
- Corriger les erreurs au fur et à mesure jusqu'à ce que le logiciel fonctionne "à peu près".
- Il n'y a pas de phases clairement définies comme l'analyse, la conception ou la maintenance.

Idée → Codage → Tests → Corrections → Codage → ...

Le modèle de cycle de vie Code-and-Fix (coder et corriger) est l'un des modèles les plus simples et anciens du génie logiciel. Il est souvent considéré comme un anti-modèle, car il présente beaucoup de limites.

Le Modèle Du Code-and-fix : Exemple

Développer rapidement un petit logiciel qui calcule la moyenne d'un étudiant.

1. Démarrage sans analyse
Le développeur ne rédige pas de cahier des charges.
Il ne précise pas :
le nombre de matières,
les coefficients,
les cas d'erreur (notes manquantes).
Il commence directement à coder.

2. Codage immédiat
Il écrit une première version du programme :

- saisie des notes,
- calcul de la moyenne,
- affichage du résultat.

3. Tests informels
Il exécute le programme manuellement.
Il découvre des problèmes :

- mauvaise gestion des coefficients,
- crash si une note est négative.

4. Corrections successives
Il corrige le code.
Il ajoute des conditions.
Il modifie la structure du programme.

Le Modèle Du Code-and-fix : Exemple

5. Nouvelles demandes → nouvelles corrections
On lui demande :

- d'ajouter la mention (Passable, Bien, Très bien),
- d'exporter les résultats.

→ Il modifie encore le code.

6. Résultat final
Le programme fonctionne,
mais le code est :

- peu structuré,
- difficile à maintenir,
- difficile à faire évoluer.

Le Modèle Du Code-and-fix

Avantages

- ✓ Très simple à comprendre
- ✓ Rapide au démarrage
- ✓ Peu de documentation
- ✓ Peut convenir pour :
  - de très petits programmes
  - des prototypes rapides
  - des projets personnels ou académiques simples

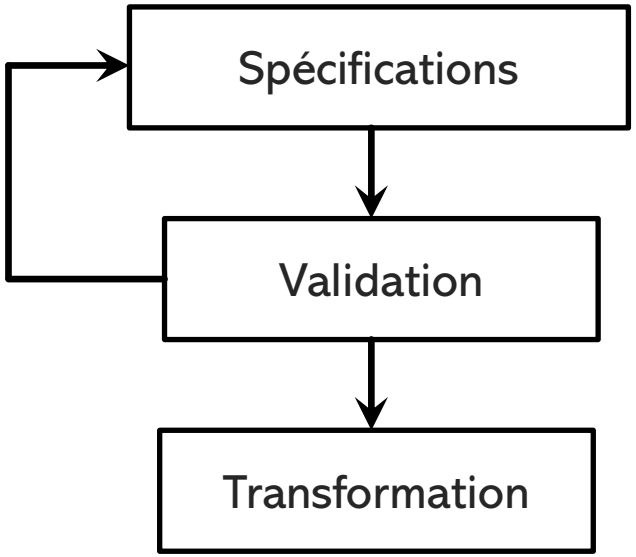
Inconvénients

- ❌ Aucune planification
- ❌ Les exigences ne sont pas clairement définies
- ❌ Qualité du code faible (code difficile à maintenir)
- ❌ Coût élevé à long terme
- ❌ Difficile à faire évoluer
- ❌ Risque élevé d'échec pour des projets moyens ou grands

→ Plus le projet grandit, plus les corrections deviennent **coûteuses et complexes.**

Le Modèle De Transformation Automatique

C’est un modèle basé sur la possibilité de transformer automatiquement des spécifications en programmes. L’essentiel de l’effort va donc porter sur une description exhaustive des spécifications qui devront être complètement validées. Une succession de cycles Phase de Spécifications – Phase de Validation s’achève par la phase de Transformation, où s’effectue la génération du code.

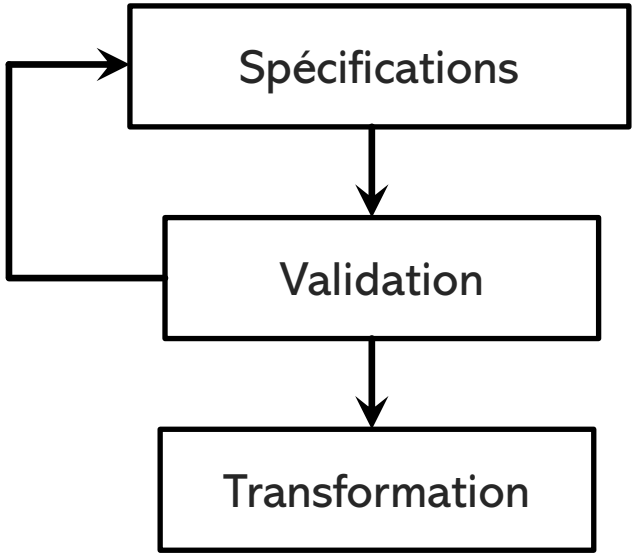


Le Modèle De Transformation Automatique

Exemple : Génération d'une application de gestion. Une entreprise décrit son système avec :un modèle UML (classes, relations),des règles métier.

À l'aide d'un outil de transformation :

- les classes Java sont générées automatiquement,
- la base de données est créée,
- une interface web de base est produite.



Le développeur n'intervient que pour des fonctionnalités avancées.

Le Modèle De Transformation Automatique

Avantages

- Réduction des erreurs humaines
- Gain de temps et de productivité
- Cohérence entre spécification et code
- Maintenance facilitée (modèle → régénération du code)

Inconvénients

- Coût initial élevé (outils, générateur)
- Spécifications difficiles à écrire
- Flexibilité limitée
- Forte dépendance aux outils

Quand l'utiliser ?

✓ Projets bien spécifiés
✓ Domaines stables
✓ Génération répétitive
✓ Systèmes critiques

❌ Projets innovants
❌ Prototypes rapides
❌ Besoins instables

Le Modèle En Cascade (Waterfall Model)

C'est un modèle qui a comme objectif majeur de jalonner (présenter sobrement) rigoureusement le processus de développement et de définir de façon précise les rôles respectifs du fournisseur qui produit un livrable et du client qui accepte ou refuse le résultat. Le découpage temporel se présente comme une succession de phases affinant celles du découpage classique :

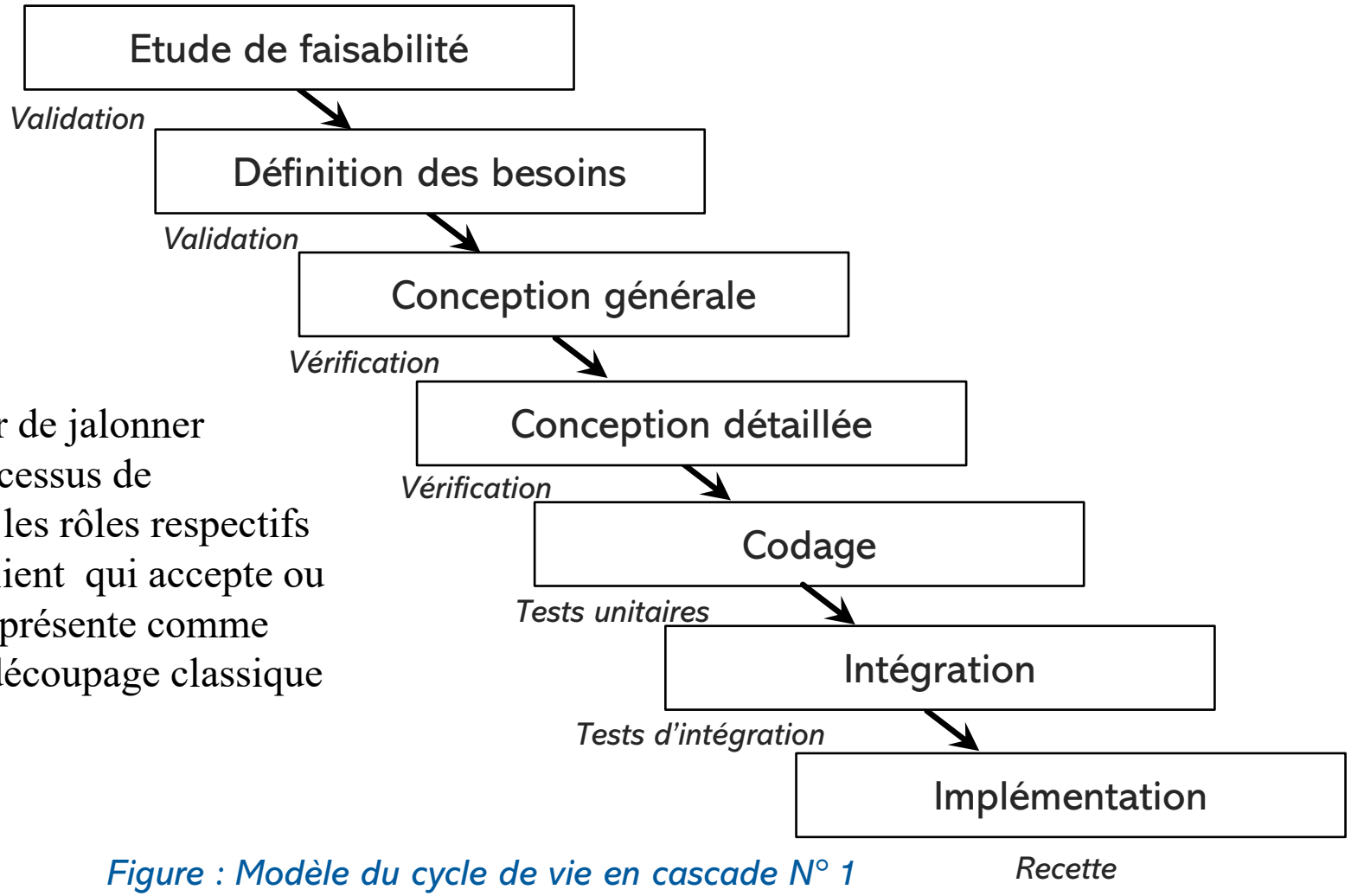


Figure : Modèle du cycle de vie en cascade N° 1

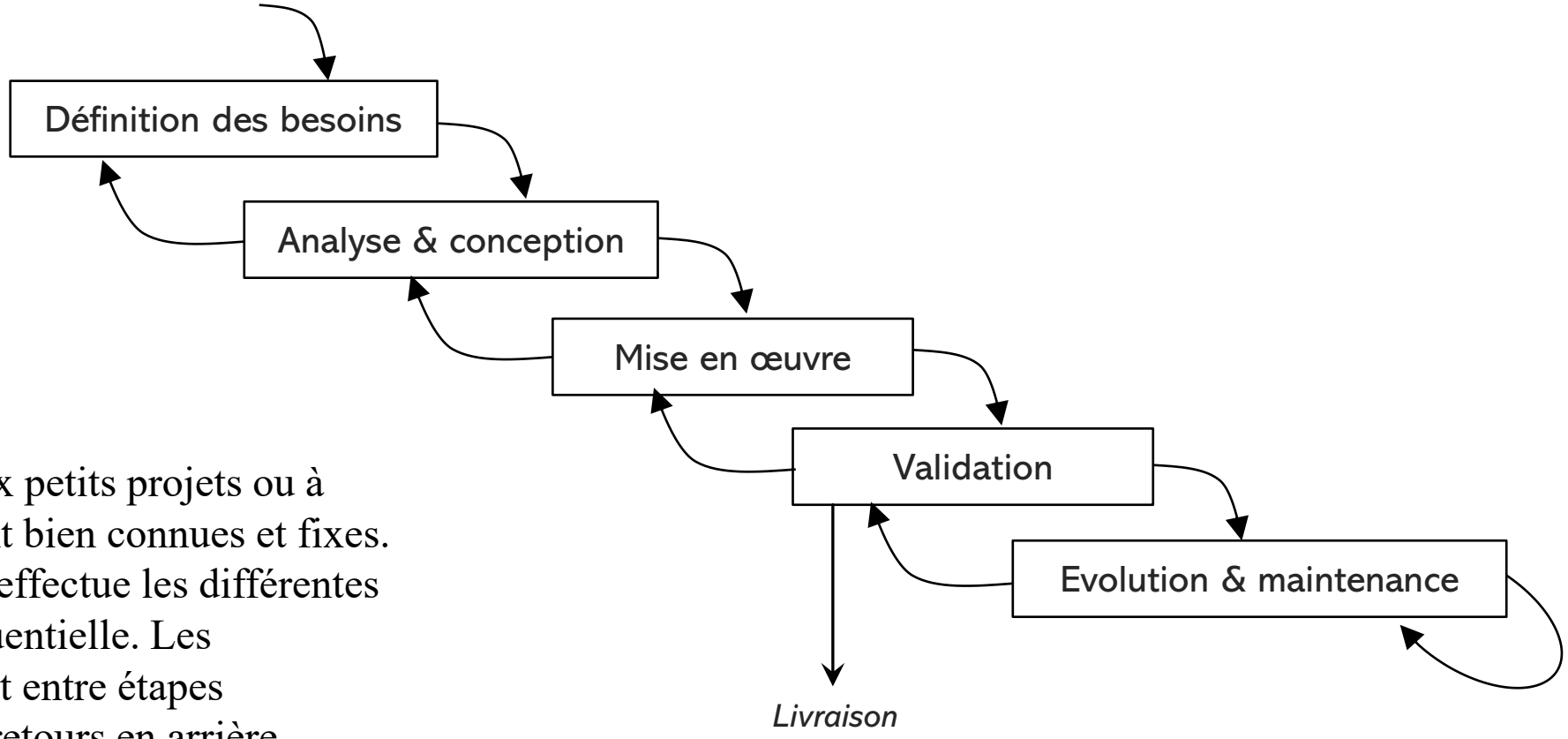
Le Modèle En Cascade (Waterfall Model)

Le cycle de vie dit de la « cascade » date de 1970, il est l'œuvre de Royce. Ce Cycle de vie est linéaire (présentoir) sans aucune évaluation entre le début du projet et la validation. Ici, Le projet est découpé en phases successives dans le temps et à chaque phase correspond une activité principale bien précise produisant un certain nombre de livrables et On ne passe à l'étape suivante que si les résultats de l'étape précédente sont jugés satisfaisants.

Ce modèle de développement est simple il est constitué d'une série d'étapes. Chaque étape se termine à une date précise par la production de certains documents ou logiciels. Le passage à l'étape suivante ne se fait que si les résultats de l'étape en cours sont jugés satisfaisants.

Il faut signaler que le modèle original ne comportait pas de possibilité de retour en arrière (l'inconvénient de ce modèle) : les erreurs de spécifications sont généralement détectées au moment des tests, voire au moment de la livraison du logiciel à l'utilisateur. Leur correction nécessite alors de reprendre toutes les phases du processus. La possibilité de retour en arrière a été rajoutée par la suite.

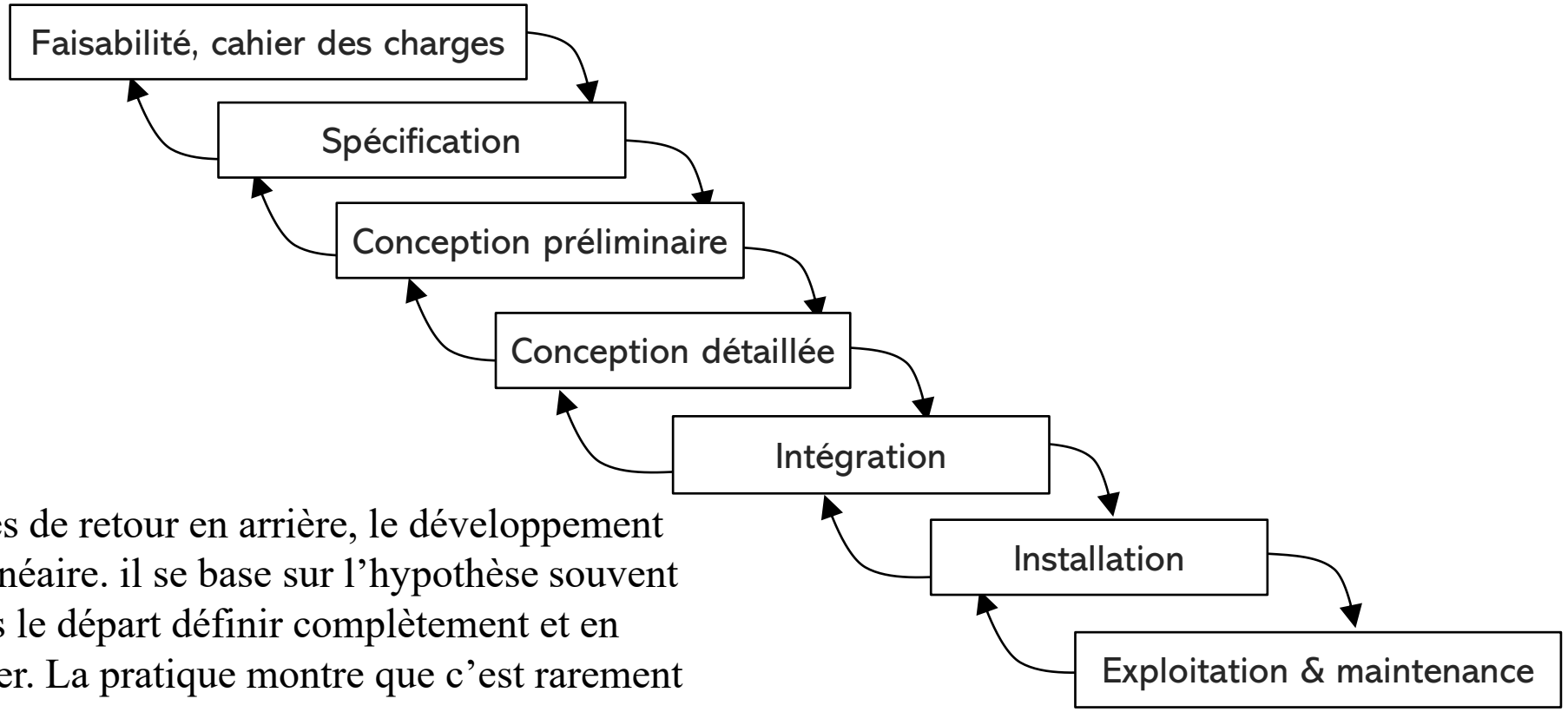
Le Modèle En Cascade (Waterfall Model)



Ce modèle est mieux adapté aux petits projets ou à ceux dont les spécifications sont bien connues et fixes. Dans le modèle en cascade, on effectue les différentes étapes du logiciel de façon séquentielle. Les interactions ont lieu uniquement entre étapes successives : on s'autorise des retours en arrière uniquement sur l'´étape précédente. Par exemple, un test ne doit pas remettre en cause la conception architecturale.

Figure : Modèle du cycle de vie en cascade N° 2

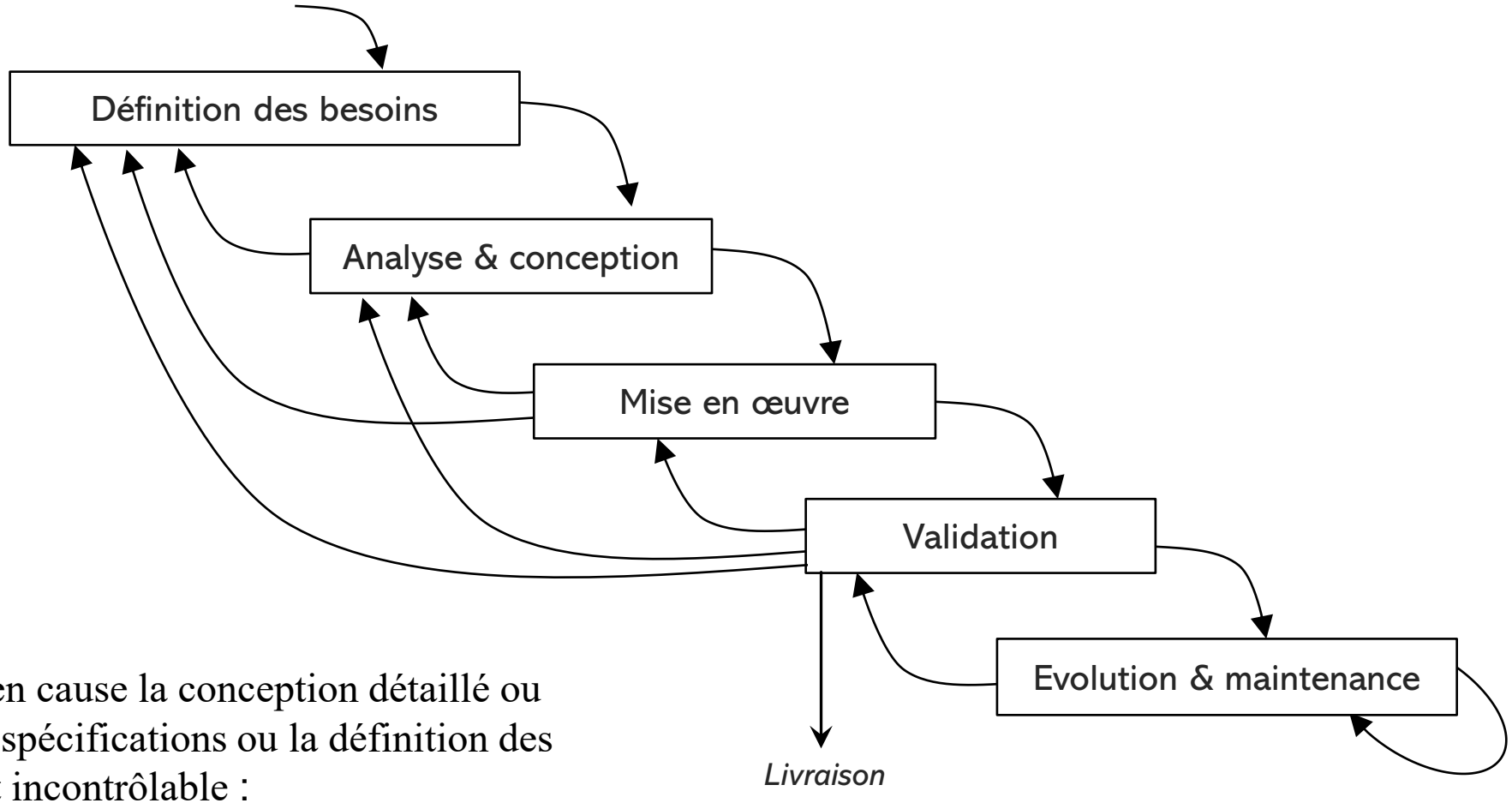
Le Modèle En Cascade (Waterfall Model)



Même avec ces possibilités de retour en arrière, le développement reste fondamentalement linéaire. il se base sur l'hypothèse souvent irréaliste que l'on peut dès le départ définir complètement et en détail ce qu'on veut réaliser. La pratique montre que c'est rarement le cas.

Figure : Modèle du cycle de vie en cascade N° 3

Le Modèle En Cascade (Waterfall Model)



Lorsque qu'un test remet en cause la conception détaillé ou architecturale ou, pire, les spécifications ou la définition des besoins, le modèle devient incontrôlable :

Figure : Modèle du cycle de vie en cascade incontrôlable

Le modèle en V

Le modèle en V du cycle de vie du logiciel précise la conception des tests : (les tests système sont préparés à partir de la spécification; les tests d'intégration sont préparés à partir de la conception architecturale ; les tests unitaires sont préparés à partir de la conception détaillée des composants). Dérivé du modèle de la cascade, le modèle en V du cycle de développement montre non seulement l'enchaînement des phases successives, mais aussi les relations logiques entre phases plus éloignées. Ce modèle fait apparaître le fait que le début du processus de développement conditionne ses dernières étapes. Le modèle du cycle de vie en V est souvent adapté aux projets de taille et de complexité moyenne.

Le modèle en V

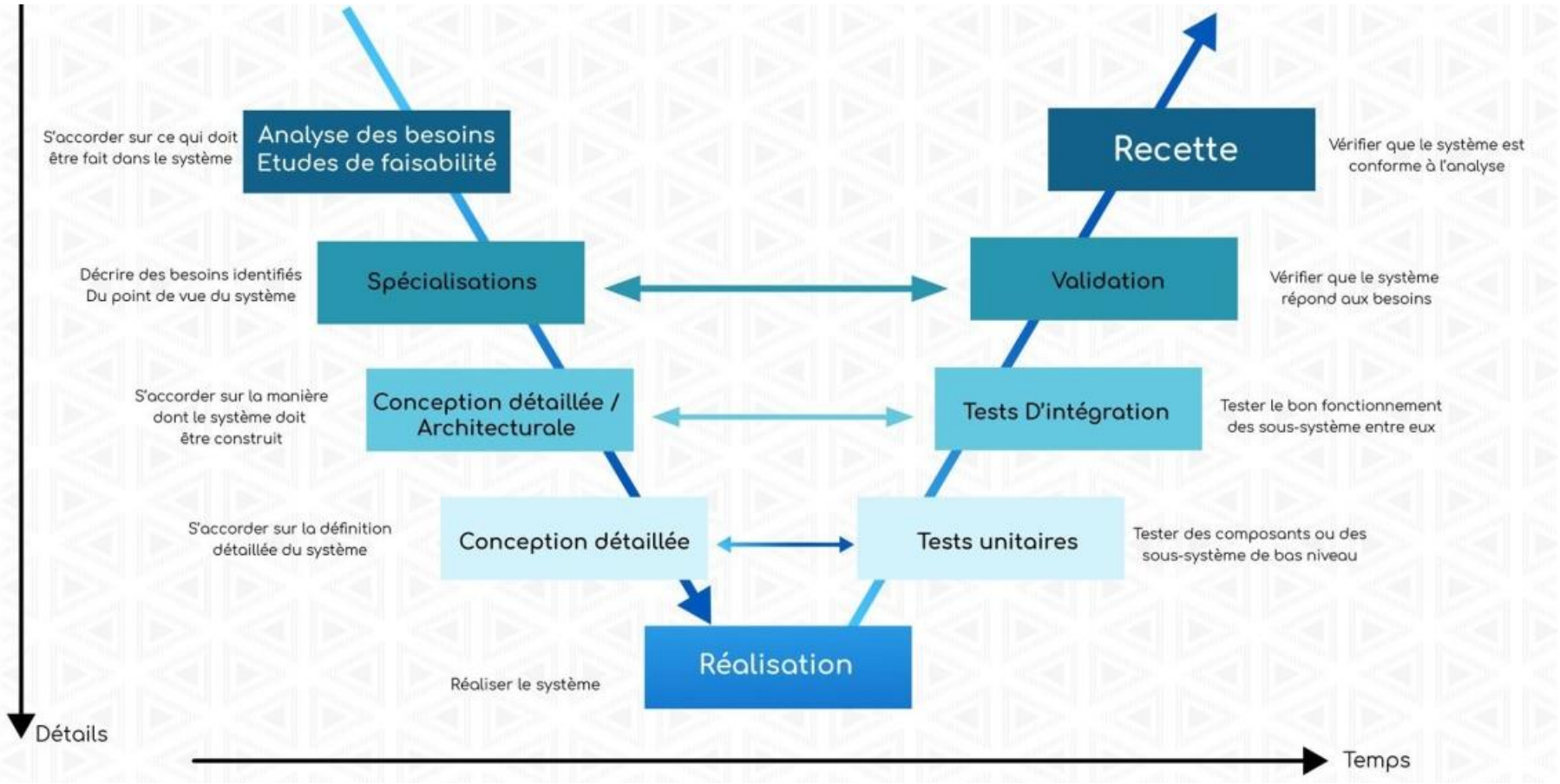
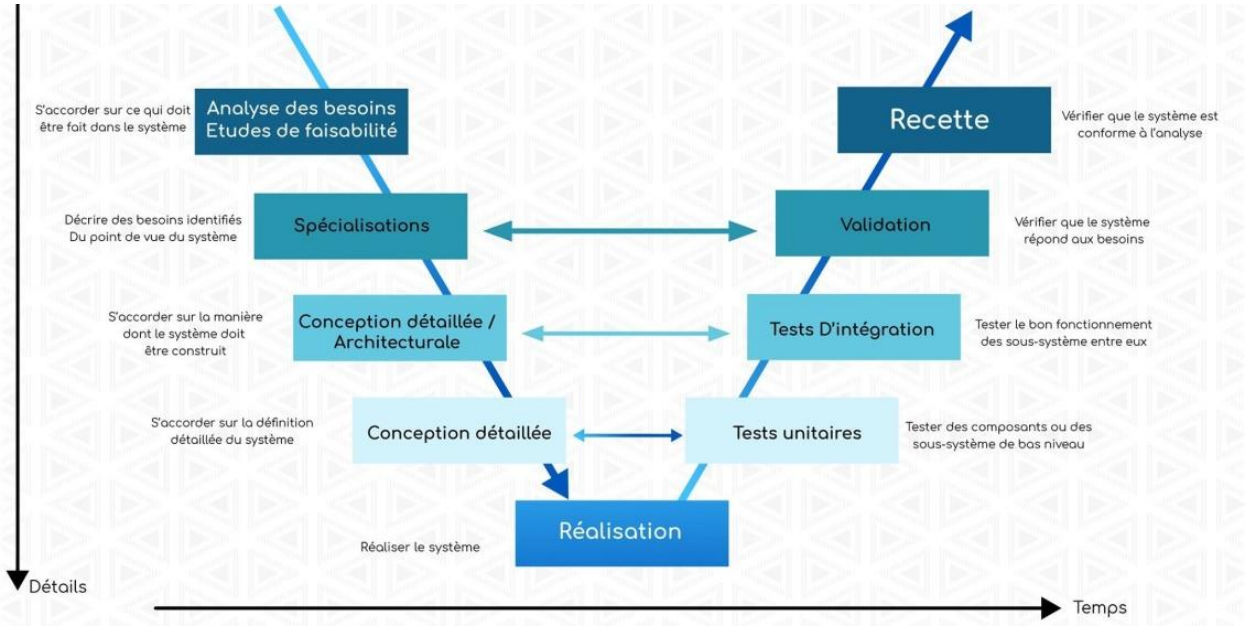


Figure : Modèle du cycle de vie en V

Le modèle en V

Le model en V présente deux branches. La première branche correspond à un modèle en cascade classique mais avec des sorties correspondant à des descriptions et des tests plus détaillés (vérification + validation). La seconde branche correspond à des tests effectifs effectués sur des composants réalisés.

- L'avantage d'un tel modèle est d'éviter d'énoncer une propriété qu'il est impossible de vérifier objectivement une fois le logiciel réalisé.
- Avec les jeux de tests préparés dans la première branche, les étapes de la deuxième branche peuvent être mieux préparées et planifiées



Le Cycle De Vie En Spirale

Ce modèle repose sur le même principe que le modèle évolutif, mais il s'inscrit dans une relation contractuelle entre le client et le fournisseur. De ce fait les engagements et validations présentent un caractère formalisé. Chaque cycle donne lieu à une contractualisation préalable, s'appuyant sur les besoins exprimés lors du cycle précédent. Un cycle comporte six phases :

- Analyse du risque ;
- Développement d'un prototype (modèle, archétype…) ;
- Simulation et essais du prototype ;
- Détermination des besoins à partir des résultats des essais ;
- Validation des besoins par un comité de pilotage ;
- Planification du cycle suivant.

Le Cycle De Vie En Spirale

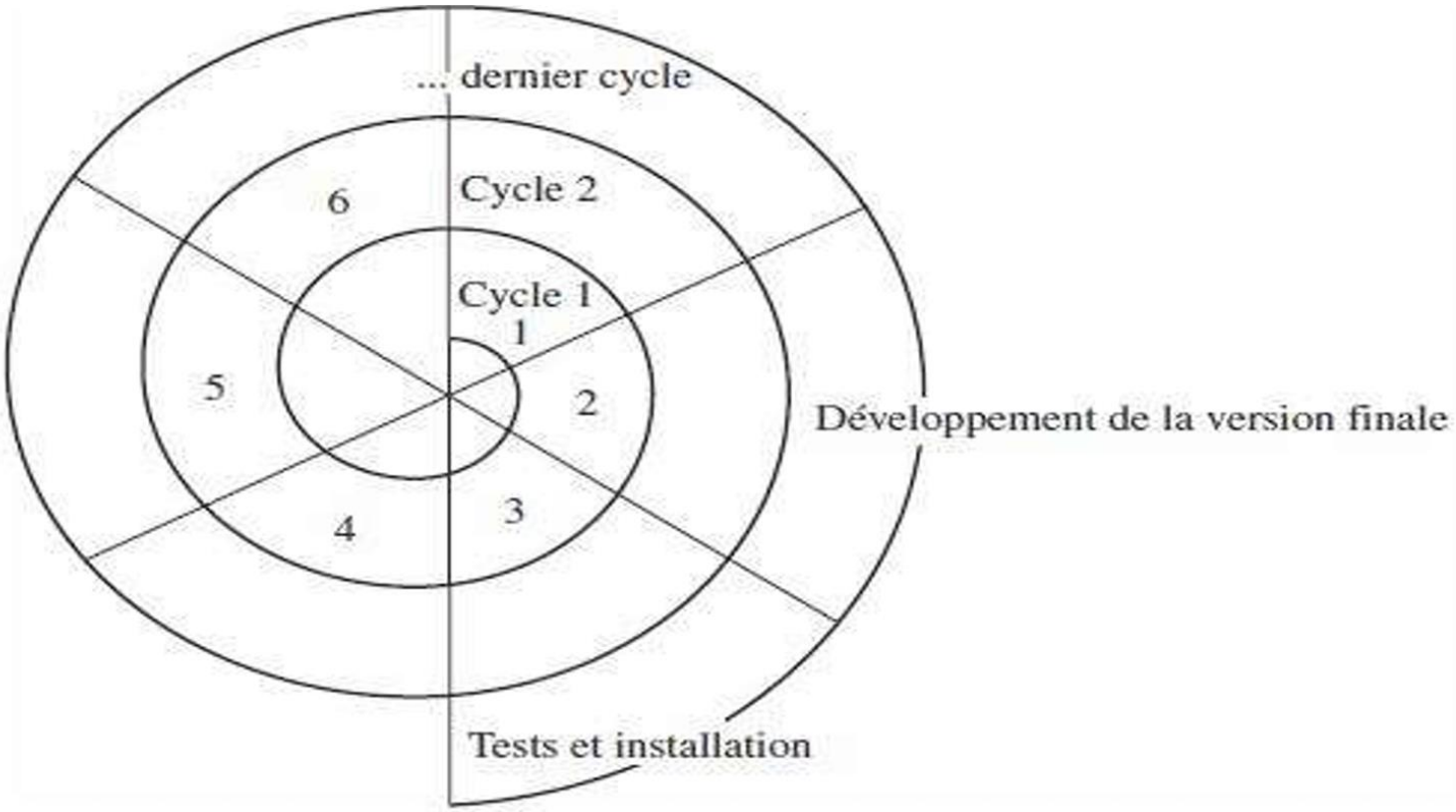


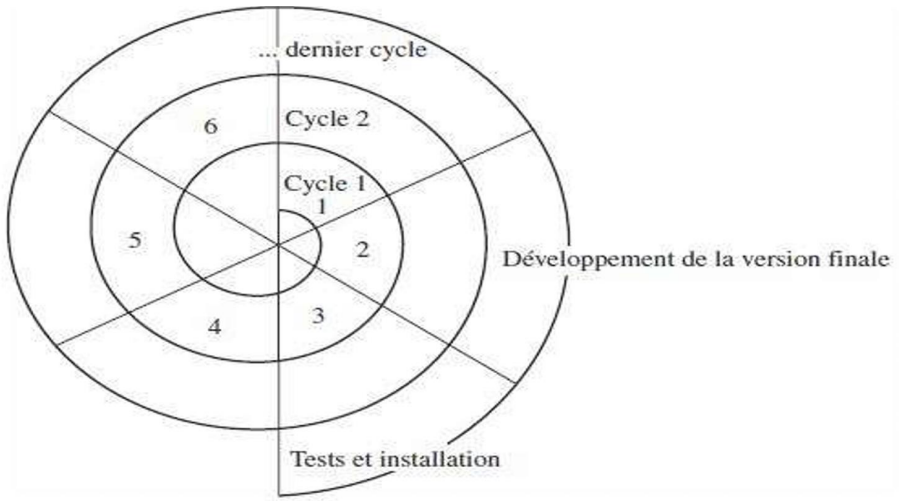
Figure : Modèle du cycle de vie en Spirale

Le Cycle De Vie En Spirale

Le modèle en spirale (proposé en 1988) insiste particulièrement sur l'analyse des risques tels que les exigences démesurées par rapport à la technologie, la réutilisation de composants, calendriers et budgets irréalistes, la défaillance du personnel ….etc.
Chaque cycle se déroule en quatre étapes :

1. Enquête et examen de la situation existante (objectifs, alternatives, contraintes) dans l'entreprise, en prenant soin de bien prendre en compte les systèmes existants. La situation doit être mesurée de façon à pouvoir évaluer les risques existants.

2. Si le poids des risques positifs est plus important, la phase suivante consiste alors à établir (planifier) ce qui doit être fait. Il peut y avoir une étape de prototypage pour éventuellement choisir entre plusieurs solutions.

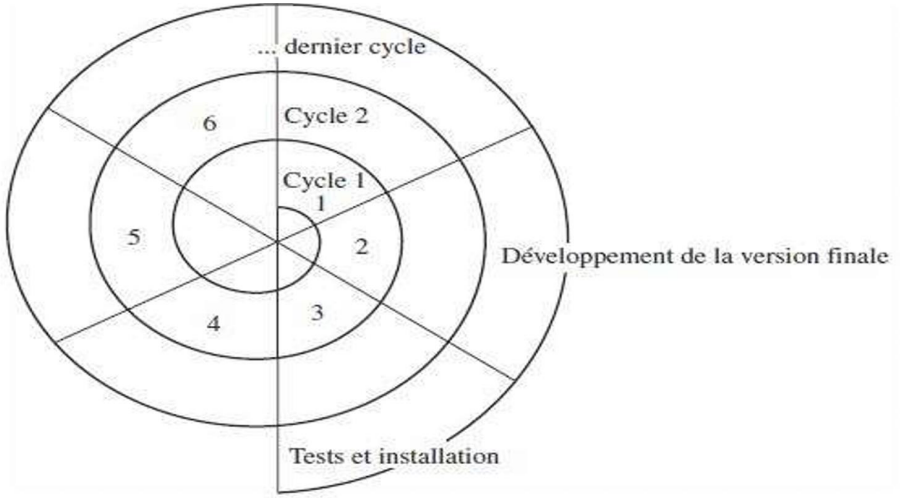


Le Cycle De Vie En Spirale

Le modèle en spirale (proposé en 1988) insiste particulièrement sur l'analyse des risques tels que les exigences démesurées par rapport à la technologie, la réutilisation de composants, calendriers et budgets irréalistes, la défaillance du personnel ….etc.
Chaque cycle se déroule en quatre étapes :

3. La réalisation de la solution nécessite la mise en œuvre d'un mini processus en cascade.

4. La situation est ensuite réévaluée pour déterminer si un développement supplémentaire est nécessaire, puis planifier l étape suivante. Ce modèle est utilisé pour des projets dont les risques sont importants.



Le Cycle De Vie En Spirale

À chaque tour de spirale, on analyse, on conçoit, on développe, on teste, puis on décide de continuer ou non.

Le Cycle De Vie En Spirale

Les 4 activités de chaque boucle (itération) de la spirale

1. Définition des objectifs

- Définir ce que l'on veut réaliser dans cette itération
- Exemples :
  - Ajouter une fonctionnalité
  - Améliorer la performance
  - Tester une nouvelle architecture

3. Développement et validation

•Concevoir, coder et tester la partie définie

•Validation par tests, démonstration ou prototype

2. Analyse et réduction des risques

- Identifier les risques techniques, fonctionnels ou organisationnels
- Exemples de risques :
  - Technologie non maîtrisée
  - Besoins mal compris
  - Problèmes de performance ou de sécurité
- Prototypage ou études pour réduire ces risques

→ **C'est l'étape clé du modèle spiral**

4. Évaluation et planification

•Le client/utilisateur évalue le résultat

•Décision :
  - Continuer
  - Modifier
  - Arrêter le projet

•Planification de la prochaine itération

Le Cycle De Vie En Spirale

Avantages

- ✓ Très bonne gestion des risques
- ✓ Flexible face aux changements
- ✓ Feedback régulier du client
- ✓ Adapté aux projets complexes et critiques

Inconvénients

- ❌ Complexe à gérer
- ❌ Coûteux
- ❌ Nécessite une forte expertise en analyse des risques
- ❌ Peu adapté aux petits projets

Le Modèle Par Prototypage

Le prototypage permet de contourner la difficulté de la validation liée à l'imprécision des besoins et caractéristiques du système à développer.

Cela veut dire que lorsqu'il est difficile d'établir une spécification détaillée, on a recours au prototypage qui est considéré, dans ce cas, comme un modèle de développement de logiciels.

Il s'agit d'écrire une première spécification et de réaliser un sous ensemble du produit logiciel final. Ce sous ensemble est alors raffiné incrémentalement et évalué jusqu'à obtenir le produit final.

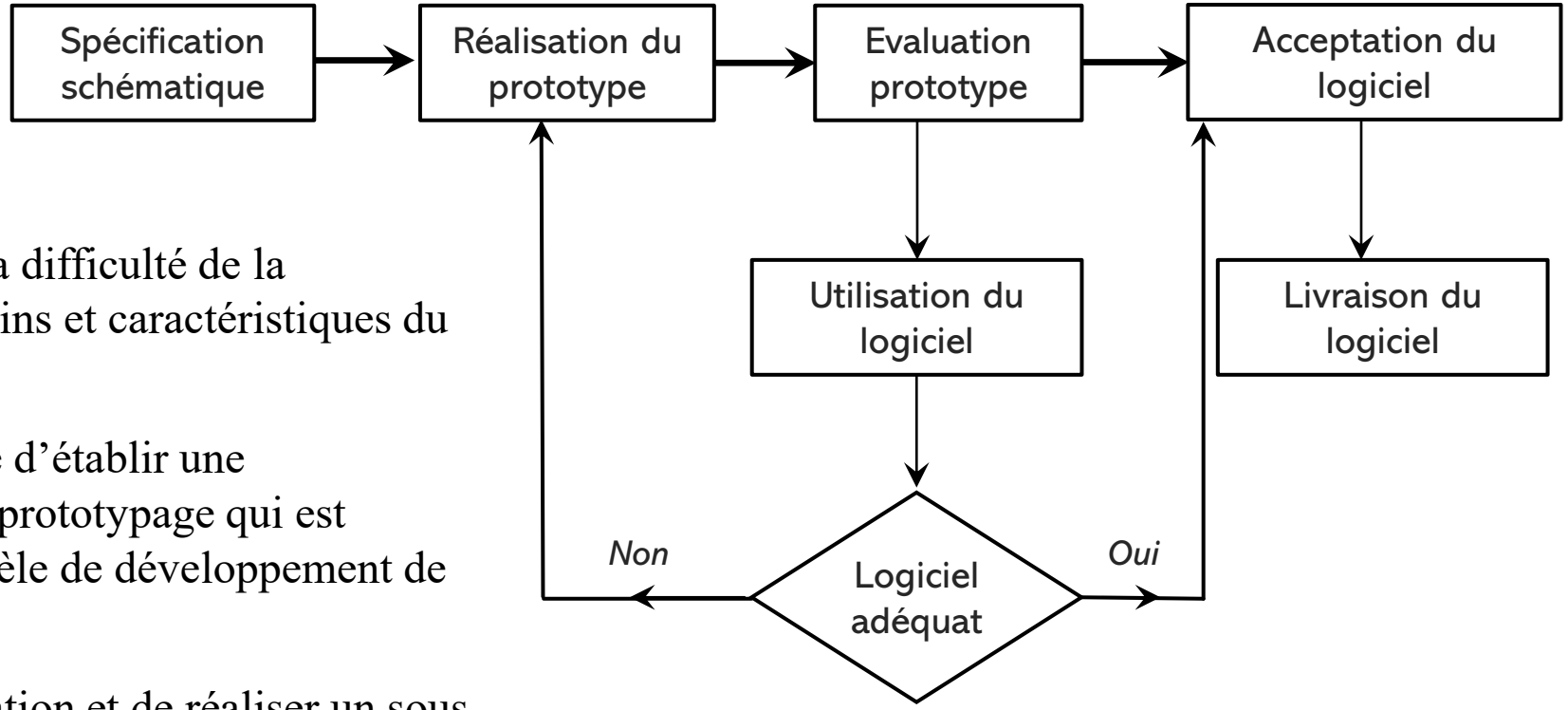


Figure : Modèle du cycle de vie par prototype

Le Modèle Par Incrément

Contrairement aux autres modèles où un logiciel est séparé en composants (développés séparément) et intégrés à la fin du processus, dans le modèle incrémental le produit est délivré en plusieurs fois, de manière incrémentale, c'est à dire en le complétant au fur et à mesure et en profitant de l'expérimentation opérationnelle des incréments précédents.

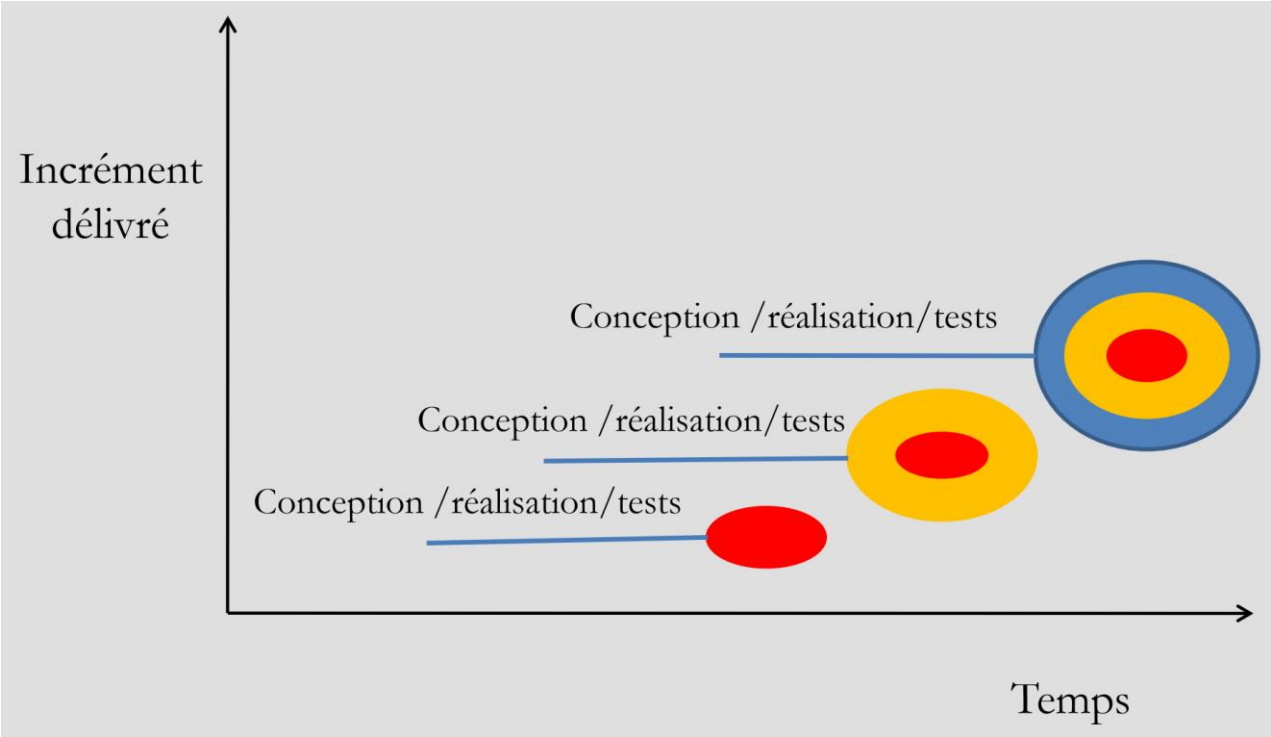


Figure : Modèle du cycle de vie par incrément

Le Modèle Par Incrément

Chaque incrément peut donner lieu à un cycle de vie classique plus ou moins complet. Les premiers incréments peuvent être des maquettes (jetables s'il s'agit juste de comprendre les besoins des utilisateurs) ou des prototypes (réutilisables pour passer au prochain incrément en les complétant et/ou en optimisant leur implantation).

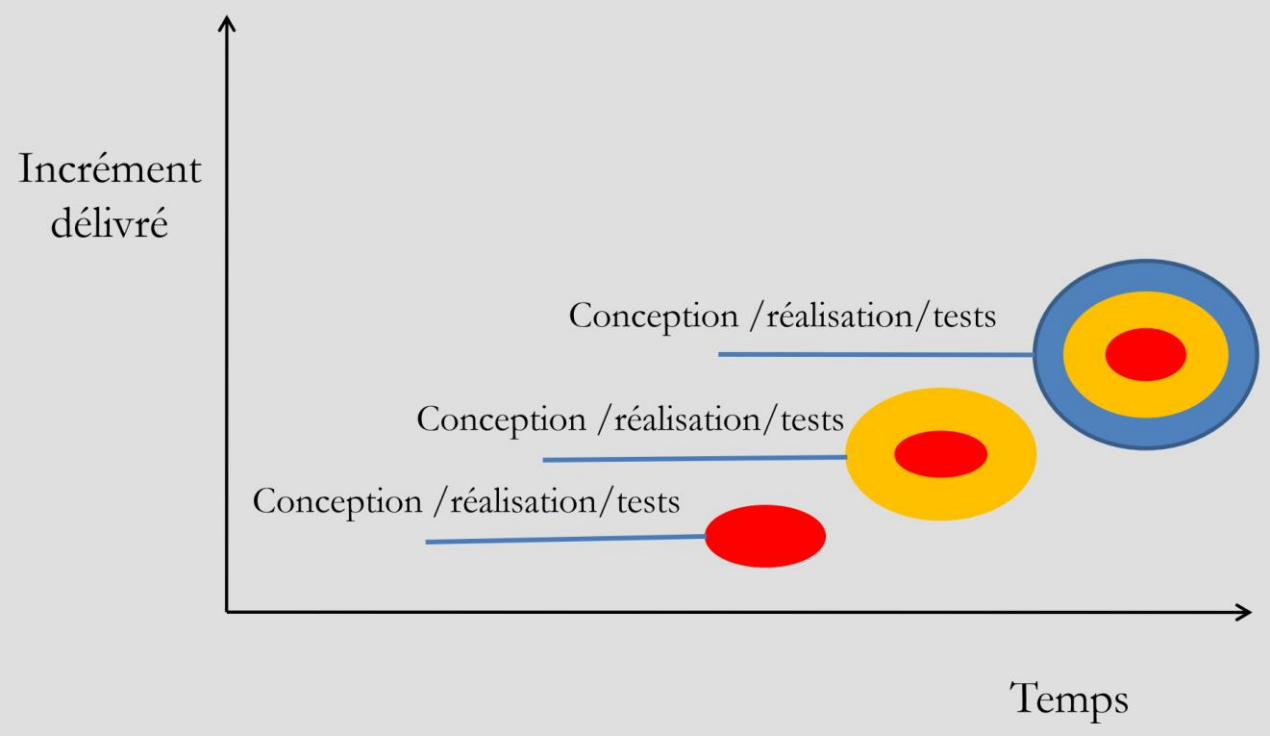
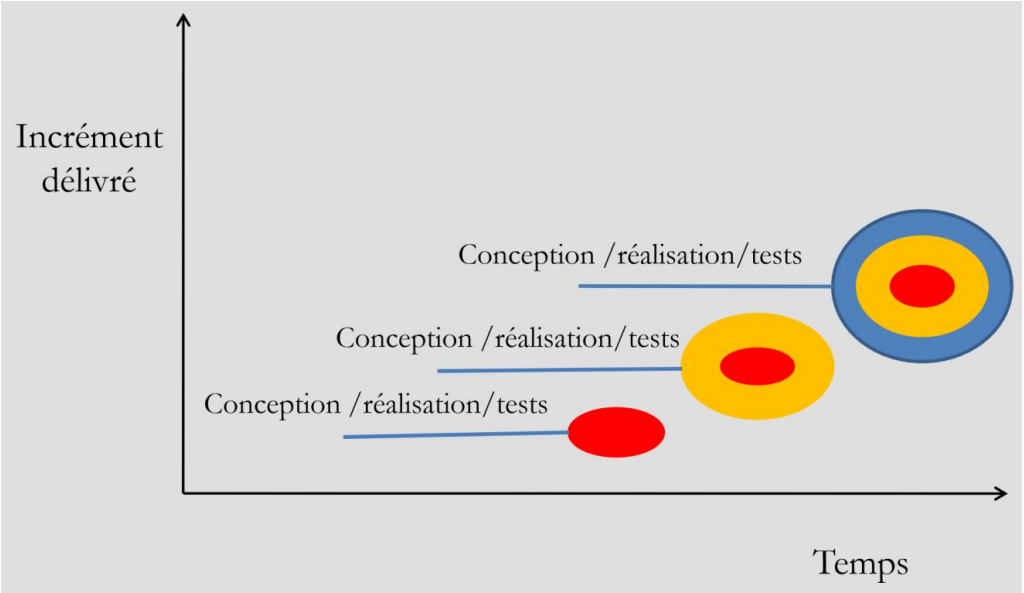


Figure : Modèle du cycle de vie par incrément

Le Modèle Par Incrément

- Le problème majeur dans ce modèle est la possibilité de remise en cause du noyau et des incréments précédents.
- De même, la possibilité de ne pas pouvoir intégrer d'autres incréments. Afin de réduire ces risques, d'une part il est nécessaire de définir les incréments au début du projet et ce de façon globale. D'autre part, les incréments devront être le plus indépendant possible, aussi bien fonctionnellement qu'au niveau de la séquence de développement.



Quel modèle faut-il utiliser?

« Il n'existe pas de modèle universel en génie logiciel »

Remarque

Il n'y a pas de modèle idéal car tout dépend des circonstances. Souvent, un même projet peut mêler différentes approches, comme par exemple le prototypage pour les sous-systèmes à haut risque et la cascade pour les sous-systèmes bien connus et à faible risque.

Quand utiliser …

Modèle code-and-fix

Quand l'utiliser ?

Très petits projets

- Prototypes rapides
- Développement personnel ou expérimental

Exemples

- Petit script

À éviter si

- Projet critique ou professionnel

Quand utiliser …

Modèle en cascade

Quand l'utiliser ?

- Les besoins sont clairs, stables et bien définis dès le départ
- Le projet est simple ou de taille moyenne
- Peu ou pas de changements attendus

Exemples

- Logiciel administratif simple
- Application avec cahier des charges figé

À éviter si

- Les besoins évoluent souvent

Quand utiliser …

Modèle en V

Quand l'utiliser ?

- Forte exigence de **qualité et de validation**
- Besoins bien définis
- Projet nécessitant beaucoup de **tests**

Exemples

- Systèmes bancaires
- Logiciels industriels ou médicaux

À éviter si

- Les besoins changent fréquemment

Quand utiliser …

Modèle en spirale

Quand l'utiliser ?

- Projet complexe
- Risques élevés (techniques, sécurité, performance)
- Besoins partiellement connus

Exemples

- Systèmes de sécurité
- Applications avec nouvelles technologies

À éviter si

- Petit projet ou budget limité

Quand utiliser …

Modèle Prototype

Quand l'utiliser ?

- Les besoins sont flous ou mal définis
- Forte interaction avec le client
- Interface utilisateur importante
- Nécessité de valider rapidement les attentes

Exemples

- Applications web ou mobiles
- Systèmes avec interfaces complexes
- Logiciels orientés utilisateurs finaux

À éviter si

- Les besoins sont déjà parfaitement définis
- Projet très critique (sécurité, temps réel) sans validation formelle

Quand utiliser …

Modèle itératif / incrémental

Quand l'utiliser ?

- Besoins évolutifs
- Livraison progressive des fonctionnalités
- Interaction fréquente avec le client

Exemples

- Applications web
- Applications mobiles

À éviter si

- Le client ne peut pas donner de feedback régulier

Quand utiliser …

Modèle agile (Scrum, XP) ???

جامعة محمد الخامس بالرباط
Université Mohammed V de Rabat